

Note: Do not edit this. download the template and start working.



Prompt Log

Team Name	Dietcoke
Website	
Domain	Travel

GUIDELINES:

1. Log all significant AI prompts used during development.
2. Mention the purpose of each prompt and the AI tool used.
3. Record important refinements, outputs, and modifications.
4. Do not fabricate or alter entries after submission.
5. Continue the log throughout the entire development process.

It's not just about the code, but how you use AI to build it.

Instructions to Participants:

- Recreate the website as a functional and interactive web experience, not just a static visual representation.
- AI tools may be used as development assistants for generating code, debugging, improving UI, implementing functionality, and solving technical problems.
- Prompts must be based on the team's own observations and analysis from Round 1.
- Follow all technical and resource restrictions specified by the organizers.
- Participants must not explicitly reveal the identity of the target website to AI.

Better prompts. Smarter builds.

Prompt	AI Tools Used	Output Received	How It was used
Handover: rebuild target = Akashavani (never name the real site). Read the prompt log, clear old entries, and start fresh from this prompt as entry #1. Follow hard rules: log every significant prompt in the PDF, ≤50-word summaries, append-only, no fabrication, ground prompts in the team's own Round-1 decode observations.	Claude Code (Opus 4.7)	Read RE_BUILD-PromptLog.pdf (6 prior entries), confirmed template layout, reset the entries table, and regenerated the PDF preserving the cover page (RE:BUILD brand, DECODE. REBUILD. EVOLVE tag, Team=Dietcoke / Website=blank / Domain=Travel, guidelines, participant instructions, footer) with only this handover as row #1.	Fresh baseline for the official RE:BUILD prompt log; every subsequent build prompt will be appended below this row under the same rules.
Scaffold the Vite project, do an initial commit and push, and create the React app basic components. Stack per TechStack.md (React 18 + TS + Vite, Tailwind, React Router v6, TanStack Query, Zustand, lucide-react). Repo: N9601/Skyscanner-Rebuild.	Claude Code (Opus 4.7)	Wrote 38 files at repo root: package.json, Vite/TS/Tailwind/PostCSS configs, index.html, README, providers with QueryClient, router with 8 routes, Layout+Header+Footer, Button+Card primitives, SearchWidget with trip-type tabs, and page stubs for Home/Flights/Stays/Cars/Trips/Alerts/Assistant/404. git init -b main, added remote, initial commit, pushed to origin/main.	Establishes the working baseline app; next prompts will layer mock APIs, real Flights results with filters, price calendar, trip plan store, and alerts on top of this scaffold.
UI inspiration screenshots from Dribbble shared; use framer-motion; repo remade, push again as initial commit with N9601 as the only contributor in everything.	Claude Code (Opus 4.7 / Fable 5)	Reset local git history, re-pushed a clean initial commit with no AI attribution trailer, installed framer-motion, and began layering animation primitives (motion variants, Chip) on the scaffold.	Fresh single-author repo baseline; design DNA from the shared shots guides all UI work.
Make commits on every small change we do.	Claude Code (Opus 4.7 / Fable 5)	Adopted small-commit workflow: each logical change (deps, tokens, data, shell, search, pages) committed and pushed separately with descriptive messages.	Keeps an auditable, granular history for judging and rollback.
UI must feel like a proper premium SaaS app, no em dashes, build the app fully end to end.	Claude Code (Opus 4.7 / Fable 5)	Built full MVP: Inter/Manrope fonts, token system, persisted theme/trips/alerts stores, premium header with animated nav, glass search bar, flights results with filters/sort/price strip, stays, cars, trips dashboard, alerts, scripted assistant.	Delivered the complete demo surface set in premium styling.
Fill the app with local and international flights using actual flight data; do we need any APIs?	Claude Code (Opus 4.7 / Fable 5)	No API needed: added real airport dataset (36 airports with coordinates), real airline names, haversine distance-based durations, realistic INR pricing per route class, and hub-aware stopovers. Live APIs conflict with the offline demo constraint.	Flight results now reflect realistic domestic and international routes offline.

Prompt	AI Tools Used	Output Received	How It was used
UI seems bland; show an actual SaaS UI taking heavy inspiration from the shared Dribbble shots.	Claude Code (Opus 4.7 / Fable 5)	Rebuilt search as a segmented glass bar with circular dark submit, added black pill CTAs, floating hero decor cluster, dark 3D tilted destination carousel with center focus, stat band, and greener callout section.	Home and search now mirror the reference visual language.
Take animation and styling inspiration from my profaxis project.	Claude Code (Opus 4.7 / Fable 5)	Ported profaxis motion language: IntersectionObserver Reveal, cursor TiltCard with sheen, CountUp, line-rise letterpress hero reveal, route-fade transitions, paper grain film, ticker tape marquee, hairline card hovers, reduced-motion guards.	App-wide motion system now matches the profaxis editorial polish.
I have a Gemini API key for the assistant chat; wire it up and make a .env file to paste the key.	Claude Code (Opus 4.7 / Fable 5)	Created gitignored .env plus .env.example, added Gemini client (gemini-2.5-flash) with travel-only system prompt; deterministic route/budget intents stay scripted, open questions go to Gemini with scripted fallback on error.	Assistant is live-LLM powered when the key is present, offline-safe otherwise.
Append the prompt log; fix overlapping containers; add actual airline logos, HD photos with animation on trending escapes; other pages are too empty, add data.	Claude Code (Opus 4.7 / Fable 5)	Added kiwi.com airline logos with monogram fallback, Unsplash photos with hover zoom on carousel cards, moved carousel info pill below the card stage, added popular routes and city grids to empty flight/stay/car states, appended log rows.	Richer content everywhere; overlap resolved; log current.
Make an actual 2D SVG logo, current one seems basic.	Claude Code (Opus 4.7 / Fable 5)	Designed a custom paper-plane mark: gradient rounded square, orbit ring, dashed contrail, layered white and blue plane. Shipped as a React Logo component in header and footer plus favicon.svg.	Distinct brand mark replaces the generic library icon.
Move the globe left so nothing is over it.	Claude Code (Opus 4.7 / Fable 5)	Shifted the hero globe decoration from center to 30 percent left in the illustration column so the floating ticket and CO2 cards no longer cover it.	Hero composition reads cleanly with no element overlap.
Change the photos, add something iconic and beautiful.	Claude Code (Opus 4.7 / Fable 5)	Verified candidate Unsplash IDs return 200, swapped in iconic imagery: Maldives overwater villas, Kashmir lake, Burj Khalifa, Wat Arun at golden hour, Chureito Pagoda with Mount Fuji; kept Goa palms and Eiffel Tower.	Trending escapes carousel now shows landmark photography.

Prompt	AI Tools Used	Output Received	How It was used
Needs a better, more minimalistic UI (assistant screenshot shared).	Claude Code (Opus 4.7 / Fable 5)	Redesigned the assistant into a minimal hairline card: compact bubbles, quiet suggestion chips shown only at start, round input with monochrome send button, subtle typing dots, centered header naming the Gemini or scripted mode.	Assistant panel matches the app's minimalist premium language.
What major features and UI should come next?	Claude Code (Fable 5)	Proposed a ranked roadmap: flight detail drawer, Everywhere search, whole-month price grid, MapLibre stays map, and a polish pass (command palette, route transitions, mobile bottom nav).	Set the build order for the remaining hackathon features.
Is Gemini connected?	Claude Code (Fable 5)	Tested the key live: it authenticated but gemini-2.5-flash returned 404 (retired for new keys). Switched the client to gemini-3.6-flash, verified a 200 response, committed the fix.	Assistant now reliably reaches Gemini in production.
Build the roadmap in this order: 5, 4, 3, 1.	Claude Code (Fable 5)	Shipped all four: Ctrl+K command palette with route jumps, animated route transitions, mobile bottom nav; lazy MapLibre stays map with price-chip pins synced to cards; whole-month fare heatmap with tercile colors; flight detail drawer with timeline, fare breakdown, CO2 vs route average.	Completed the feature set for the demo, each pushed as its own commit.
Fix UI, don't leave any cards empty across the site, use proper photos.	Claude Code (Fable 5)	Raised search widget stacking so the autocomplete dropdown sits above content, added verified Unsplash photos to popular city cards and all stay cards with gradient fallbacks and hover zoom.	Every card surface now carries real imagery; dropdown overlap resolved.
Clipping text (hero screenshot shared).	Claude Code (Fable 5)	Line-rise mask was cutting descenders on 'journey'. Added padding-bottom with negative margin compensation to .line-mask so letterforms render fully while the reveal clip still works.	Hero headline renders cleanly with the letterpress animation intact.
What next to make it a fully fledged app?	Claude Code (Fable 5)	Recommended roadmap: Everywhere search, booking flow simulation, round-trip results, Supabase auth and sync, detail pages, alert loop, assistant deep links, tests and Vercel deploy.	Set the closing scope for the submission.

Prompt	AI Tools Used	Output Received	How It was used
Fix nav badge centering and seal the deal; UI still bland, needs over-the-top animation and less white; Supabase URL and publishable key shared.	Claude Code (Fable 5)	Centered the trips badge, built Everywhere search grid with filters, simulated booking handoff (contact, review, provider reference), aurora background, gradient headline and stats, shine CTAs, scroll progress bar, Supabase email auth with account menu and cloud sync of trips/alerts plus RLS schema script.	App is feature complete with real auth and a richer visual identity.
Fast, last 10 minutes.	Claude Code (Fable 5)	Added vitest with 13 passing tests over the mock API and intent router, Vercel SPA rewrite config, committed and pushed everything, appended final prompt log rows.	Submission-ready repo with tests and deploy config.

Better prompts. Smarter builds.